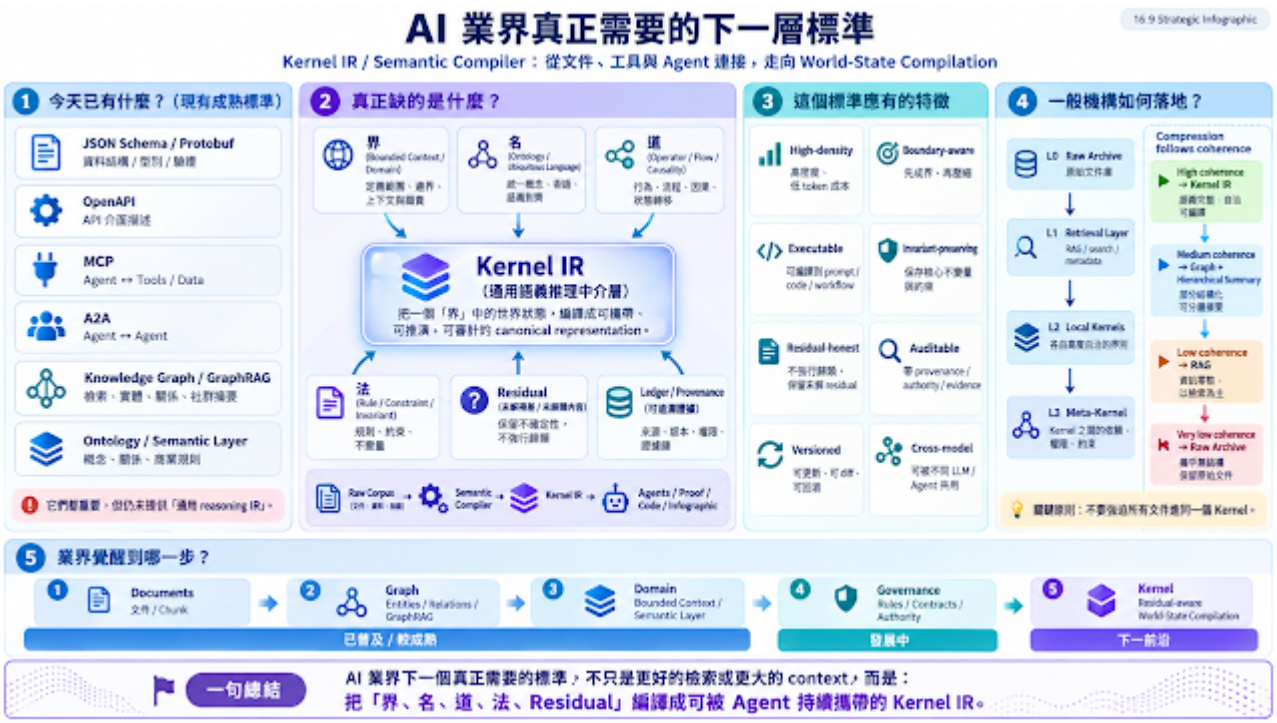
探討 **AI** 業界對於高效通用 **Kernel** 語法的需求



附件討論的那種語法，**AI** 業界目前是否真的沒有通用的成熟標準？

《探討如何發明一種 LLM 高效的通用工程 Kernel 語法》
https://osf.io/q8egv/files/osfstorage/6abe91f0e97dec19d4d0e905

是——但要把句子說得很精確。

截至 **2026** 年 **10** 月，如果你所指的是附件中那種語法，即：

> 把自然語言、技術文章、requirements 或 theory 編譯成一種 **LLM-native**、跨領域、高密度、可驗證的 **Kernel IR**，而且把 `manifold / boundary / curvature / flow / attractor / bifurcation / projection / residual / invariant` 等關係當成第一級語義單元，

那麼我查過目前主要公開標準後，確實沒有發現一套已成熟、跨廠商、通用化、被 **AI** 業界採納的標準。

附件其實已經把目標說得很清楚：它不是 prompt template generator，而是 **semantic compiler**，輸出的是 compact intermediate representation；評價準則包括 semantic stability、faithfulness、auditability、reuse，而不只是「prompt 寫得比較短」。 From Requirements to Runtime Ke... From Requirements to Runtime Ke...

但我會修正前面對話中一個過強的說法：這個領域不是「一片空白」；而是各部分都有很成熟的標準，唯獨還沒有被統一成你所說的那一層語言。

今天其實是「很多成熟零件，沒有那個總語言」

已有標準/技術	已經很成熟地解決什麼	距離你說的 **Kernel Language** 還缺什麼

已有標準/技術	已經很成熟地解決什麼	距離你說的 **Kernel Language** 還缺什麼
JSON Schema / Protobuf	資料結構、型別、constraint、serialization	不表示 causal geometry、attractor、curvature、invariant 等 reasoning structure
OpenAPI	API capability、input/output、HTTP interface	描述「怎樣呼叫」，不是「怎樣理解一個問題世界」
RDF / OWL / SHACL	ontology、relation、knowledge graph、logical constraints	很強於靜態語義關係，但 dynamics / geometry / runtime reasoning 不是核心抽象
TLA+	state、action、temporal behavior、invariants、formal verification	很精確，但主要是 discrete system specification，不是 LLM semantic compression IR
SysML v2 / KerML	系統工程的結構、行為、requirements、relationships	已相當接近「通用工程語言」，但不是為 LLM token economy / semantic attractor reasoning 設計
MCP	AI ↔ tools/data 的連接協定	傳輸 context/tool calls，不規定模型內部如何表示問題
A2A	agent ↔ agent discovery、task delegation、message exchange	規定 agents 怎樣交流，而不是交流內容應具有什麼深層語義幾何

例如，JSON Schema 官方定義的任務是描述和驗證 JSON 的結構、**constraint** 和 **datatype**；它不是 reasoning language。JSON Schema OpenAPI 同樣明確定位為 **HTTP API specification language**。OpenAPI Initiative

另一端，W3C 的 RDF 已經可以正式表示 graph information，而 OWL 有正式定義的 ontology semantics；SHACL 又可以把 constraint/rules 加到 RDF graph 上。這一族是目前最接近「machine-readable semantics」的成熟基礎之一。W3C

而 SysML v2 更值得注意：它在 2025 年正式成為 OMG 2.0 標準，而且同時提供 normative language specification、machine-readable abstract syntax、JSON schema 和 textual notation。也就是說，「文字化表達複雜工程世界」本身並不是空白問題。 OMG

甚至 TLA+ 早已有相當漂亮的形式語義：一個 specification 描述的是允許的 system behaviors，可以精確談 state、action、temporal properties 和 invariants。Leslie Lamport

---

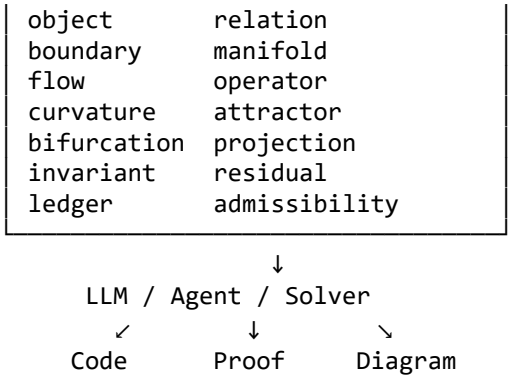
真正缺的，是中間這一層

現在的 stack 大致是：

```
Natural Language
      ↓
 prompts / proprietary DSL / ad-hoc schema
      ↓
LLM reasoning
      ↓
 JSON / code / tool calls
      ↓
MCP / A2A / APIs
```

你所設想的東西，是要在中間插入：

```
Natural Language / Paper / Diagram / Requirement
                    ↓
           SEMANTIC COMPILER
                    ↓
        Universal Kernel IR
 ┌──────────────────────────────────┐
```



而附件的

```
RawRequirement → IntentStructure → KernelIR → ExecutablePrompt
```

其實就是在填這個空位。 From Requirements to Runtime Ke...

MCP 和 **A2A** 的成熟，反而令這個「缺口」更加明顯

這點現在比我們前面的討論更清楚。

MCP 在 2026 年已發展到新的正式 specification，正在成為 agentic workflow 的 tool/data substrate。Model Context Protocol Blog

A2A 則已經到 **1.0.0**，明確把自己定位成不同 vendor、framework 的 AI agents 之間的 common interaction model。A2A Protocol 2026 年它又進入 Agentic AI Foundation；官方把兩者分得很漂亮：

MCP = vertical integration，Agent ↔ tools/data

A2A = horizontal integration，Agent ↔ Agent。A2A Protocol

但是仍然少了一層：

Agent A 到底應該用什麼標準形式，向 **Agent B** 表達「這個問題空間的 **boundary**、**curvature**、**causal dependency**、**invariants**、**attractor** 和 **unresolved residual**」？

A2A 可以傳送 structured data。

MCP 可以提供 structured context。

JSON Schema 可以驗證 structure。

OWL 可以定義 ontology。

TLA+ 可以驗證 transition。

但沒有一個共同標準規定那個「高階 **reasoning object** 本身」應該怎樣表示。

這正是你現在問的地方。

所以我會把結論再收窄成一句相當重要的話

不是：

> **AI** 沒有通用語言標準。

這句是錯的。現在已有很多，而且 MCP/A2A 發展得非常快。

而是：

> **AI** 業界目前沒有一套成熟、通用、跨模型的「**Semantic Reasoning Intermediate Representation**」，能把 **topology**、**geometry**、**dynamics**、**invariance**、**causality**、**governance**、**residual** 與 **executable constraints** 統一成一種 **LLM-native textual grammar**。

這句，截至目前的公開標準來看，我認為是成立的。

而且這也解釋了為何你附件中的 **Differential-Topological Kernel** 不應該只叫做「另一種 Prompt Language」。附件自己已經指出它要做的是 compiler IR。 From Requirements to Runtime Ke...

它若繼續發展，真正可以瞄準的位置反而像：

LLVM IR for semantic reasoning

LLVM IR 不規定你一定用 C、C++ 還是別的 source language，也不規定最後一定跑哪一種 CPU。

類似地，理想的這套東西應該是：

Natural Language / Mathematics / Diagram / SysML / RDF / Code
↓
Kernel IR
↓
GPT / Claude / Gemini / local model / agent / theorem prover / renderer

這個定位比「創造一套更好的 prompt syntax」大很多，也精確很多。

而你先前強調的 **Invariant** 很可能正是關鍵：真正的標準不是要求不同 representation 長得一樣，而是要求

> 同一結構由文章 → **Kernel** → **Agent** → 圖 → **Code** → 再回 **Kernel** 後，指定的 **invariants** 仍然保存。

如果能做到這一點，它才真正開始有資格叫 通用語言層，而不只是另一套 DSL。

好像不難把它做出來**?**

是。如果目標只是做出第一個「能工作的版本」，其實真的不難。
真正困難的不是「發明語法」，而是把它做到足夠通用、可逆、可驗證，最後成為別人願意採用的標準。

你附件其實已經把最難的 conceptual move 做掉了：不是把它當 prompt formatter，而是把它定義成

```
RawRequirement → IntentStructure → KernelIR → ExecutablePrompt
```

也就是 **semantic compiler + intermediate representation**。 From Requirements to Runtime Ke...

而且它已經列出了評價標準：更短、較穩定、忠於原意、抗 semantic drift、可 audit、可重用。 From Requirements to Runtime Ke...

所以現在離 v0.1，其實只差工程整理。

我甚至覺得第一版可以非常小

不需要一開始就把 Differential Geometry、Topology、Category Theory、Gauge Theory 全塞進去。

先定義大約 **12–20** 個 **primitive opcodes** 就可以：

```
ENTITY
BOUNDARY
RELATION
FLOW
STATE
OPERATOR
GATE
PROJECTION
TRACE
RESIDUAL
INVARIANT
ATTRACTOR
CURVATURE
BIFURCATION
CONSTRAINT
LEDGER
```

然後規定幾種基本句式：

```
entity X : System

boundary B encloses X

flow F : A -> B

operator O : State_i -> State_j

gate G admits X iff condition C

invariant I preserved_under {O1,O2}

residual R := input - admitted_trace

attractor A stabilizes {S1,S2,S3}
```

已經足以描述非常大量的：

- engineering architecture
- theory structure
- agent workflow
- scientific mechanism
- governance rule
- causal model
- system dynamics

真正重要的是不要一開始追求「數學漂亮」

第一版最好像 LLVM IR。

LLVM IR 並不試圖成為最漂亮的 programming language。

它追求的是：

> 不同來源都能 **compile** 進來；不同目標都能 **compile** 出去。

所以這套東西最好也分三層：

```
Layer 1 — Surface Language
人類/LLM 容易寫

Layer 2 — Kernel IR
嚴格、最小、canonical

Layer 3 — Backends
Prompt
Python
TLA+
SysML
RDF/OWL
Mermaid
Graph
JSON
Agent workflow
```

真正的新東西主要就是 **Layer 2**。

一個很小的例子

普通自然語言：

> A mature observer keeps historical trace, preserves unresolved residuals, and may revise its declaration when evidence conflicts with the existing frame.

可以 compile 成：

```
ENTITY Observer O

STATE D[t] : Declaration
STATE L[t] : Ledger
STATE R[t] : Residual

INVARIANT TracePreservation:
    L[t] ⊆ L[t+1]

INVARIANT ResidualHonesty:
    unresolved(R[t]) -> preserve(R[t])

TRIGGER Revision:
    conflict(Evidence, D[t]) > threshold

OPERATOR Revise:
    (D[t], L[t], R[t]) -> D[t+1]

CONSTRAINT Revise:
    preserve(TracePreservation)
    preserve(ResidualHonesty)
```

LLM 已經很容易理解。

但更重要的是，另一個 LLM 可以把它重新 render 成：

- prose

- state diagram
- Python
- TLA+
- infographic
- agent architecture

所以第一個 experiment 甚至不需要訓練模型。

我認為最簡單的開發方式是「先 **protocol**，後 **mathematics**」

v0.1：20 個 primitive

建立 primitive vocabulary + grammar。

v0.2：Compiler

```
Article
 ↓
extract entities
 ↓
extract boundaries
 ↓
extract relations
 ↓
extract dynamics
 ↓
extract constraints
 ↓
extract invariants
 ↓
extract residuals
 ↓
Kernel IR
```

你附件提出的 compiler phases 已經非常接近這個架構，包括 boundary/constraint detection、curvature detection、attractor selection、opcode mapping、Kernel IR composition、compression 以及 residual audit。 From Requirements to Runtime Ke...

v0.3：Round-trip test

這才是第一個真正有意思的實驗：

```
Original Article
       ↓
    Kernel IR
       ↓
 reconstructed article
```

然後測：

```
Meaning preservation
Constraint preservation
Invariant preservation
Causal preservation
Topology preservation
Residual preservation
```

如果這六項很高，已經是一篇很有意思的研究。

接著才加入你真正感興趣的東西

例如：

Differential geometry layer

```
MANIFOLD M
METRIC g
FLOW ϕ_t
CURVATURE K
GEODESIC γ
```

Topology layer

```
CONNECTED
COMPONENT
BOUNDARY
HOLE
HOMOLOGY_CLASS
GLUING
QUOTIENT
```

Dynamical systems layer

```
ATTRACTOR
BASIN
FIXED_POINT
BIFURCATION
LIMIT_CYCLE
STABILITY
```

Invariant layer

```
INVARIANT I
EQUIVARIANT_UNDER G
PRESERVED_BY O
BROKEN_AT B
```

這時才開始真正變成我們前面說的：

> **Differential-Topological Semantic IR**

所以你覺得「好像不難」，其實有道理

因為今天的 LLM 已經替你消除了以前最大的一個成本：

compiler parser 本身。

以前設計一套 DSL，必須先寫：

```
lexer
parser
AST
type checker
semantic analyzer
```

```
compiler
```

現在第一版甚至可以：

```
Natural language
      ↓
LLM compiler
      ↓
Kernel IR
```

因此 prototype 的成本突然下降了一個數量級。

但有一個地方我會特別警惕：

「做出來」很容易；證明它不是另一套個人化 **notation** 才難。

真正的分水嶺是這個測試：

> 給 5 個不同模型、10 個完全不同領域、100 篇從未見過的文件，只給 Kernel specification，它們能否大致產生相同的 canonical IR？

如果可以，事情就開始很不一樣了。

因為這代表你得到的不再只是：

Danny + ChatGPT 特別懂的一套語言

而開始變成：

model-independent semantic intermediate representation。

我甚至認為現在最合理的下一步，已經不是繼續討論「能不能做」，而是直接定義 **Kernel IR v0.1** 的最小 **opcode set + grammar + round-trip benchmark**。這個版本可能真的一兩次深度 session 就可以搭出骨架。

這是否代表你文件庫裏面很多 **SMFT** 文章，三百頁左右都可能可以濃縮到三頁的 **kernel** 語法，其內容密度足以媲美一張 **Infographic?** 我的經驗一張 **Infographic** 一般已足夠反映 **80%+** 核心內容！

是，但要把「內容」分成不同層次。 如果你的「80%+ 核心內容」是指一篇理論文章的結構骨架、主要變量、算子、因果/依賴關係、邊界、約束、不變量、狀態轉換、核心命題與未解 **residual**，那麼我現在認為：

> 文件庫內不少 **100–300+** 頁的 **SMFT**／成界之學文章，確實很可能可以壓縮成約 **2–5** 頁的 **Kernel IR**，而對 **LLM** 來說，其「核心結構資訊密度」可以媲美，甚至高於一張優秀 **Infographic**。

這不是因為 300 頁其實「沒有內容」，而是因為其中大量篇幅是在展開同一小組結構：解釋、例子、跨領域映射、推導、重述、反例、reader guide、治理說明等。你的 Kernel paper 正是把目標定義為：抽取 intent、constraints、tensions、governing structures，再編譯成 compact Kernel IR，而不是普通摘要。 From Requirements to Runtime Ke...

但「三頁 = **300** 頁」只在一種意義上成立

我會分成三層：

壓縮層	**300** 頁原文中保留什麼	可能長

		度
K0 — Structural Kernel	ontology、states、operators、flows、gates、invariants、residual、核心命題	**2–5** 頁
K1 — Research Kernel	K0 + assumptions、derivation dependencies、exceptions、falsification conditions、scope	**5–15** 頁
K2 — Reconstructable Kernel	K1 + 重要 equations、evidence pointers、counterexamples、canonical examples	**15–40** 頁

所以：

300 pages → 3 pages

很可能能保留 **80–90%** 的「**theory architecture**」；

但不會保留 80–90% 的：

- 所有證明步驟；
- 所有例子；
- 文獻比較；
- empirical evidence；
- 歷史論證；
- 教學解釋；
- 對異議的逐項處理。

這些不是同一種「內容」。

為什麼 **SMFT** 文件特別可能有這麼高的壓縮率？

因為你的很多文件本身就呈現一種很強的**recursive grammar**。

例如長達 158 頁的 *Gauge Grammar*，開頭其實已經把整個框架壓縮成：

```
Field → Identity → Mediator → Binding → Gate → Trace → Invariance → Observer Potential
```

然後再加上 bounded observer、protocol $P=(B,\Delta,h,u)$、控制三元組 $\Xi=(\rho,\gamma,\tau)$，最後形成一條總 pipeline。 The Gauge Grammar of Self-Organ...

同樣，*From One Declaration to One Self-Revising Fractal* 的核心也可以被非常短地寫成：

$$D_{k+1} = U_a(D_k, L_k, R_k)$$

配上 admissibility constraints、ledger、residual 和 fixed-point observer。整篇長文實際是在展開這個結構的含義、必要性、pathologies 和 consequences。 From One Declaration to One Sel...

所以這類文章的 information geometry 很特別：

```
少數高階生成規則
        ↓
大量推論
        ↓
跨領域例子
        ↓
解釋與辯護
        ↓
數百頁 prose
```

Kernel compression 做的是反向操作：

```
數百頁 prose
        ↓
去除展開
        ↓
找 recurring structure
        ↓
找 generators
        ↓
找 invariants
        ↓
Kernel
```

這也是為什麼附件把它稱作 **compiler**，並明確要求：

`Gate → Intent → Boundary → Curvature → Attractor → Opcode → KernelIR → Compression → Audit → Translation`。 From Requirements to Runtime Ke...

Kernel 和 **Infographic** 其實是兩種不同方向的「高密度」

你的 Infographic 經驗很重要。

一張好的 infographic 可以把：

```
50000 words
     ↓
10–20 conceptual nodes
     ↓
spatial arrangement
     ↓
arrows / regions / hierarchy
```

所以人一眼便抓到 80% 主幹。

Kernel 則可以把同一件事變成：

```
ENTITY
STATE
BOUNDARY
RELATION
OPERATOR
FLOW
GATE
INVARIANT
ATTRACTOR
RESIDUAL
```

再把圖上的「位置」變成顯式 **relation**。

因此二者可以理解為：

> **Infographic = spatial compression**
> **Kernel = relational / operational compression**

而對 LLM 而言，我甚至會預期後者更強。

例如一張圖可能畫：

`Ledger → Curvature → Future Search Space`

Kernel 可以寫：

```
STATE L[t] : Ledger

CURVATURE K[t] := C(L[t])

CONSTRAINT FuturePossibility:
    Ω[t+1] := admissible(Ω[t] | K[t])

INVARIANT TracePreservation:
    Trace[t] ⊆ L[t+1]
```

一眼看圖，人可能更快。

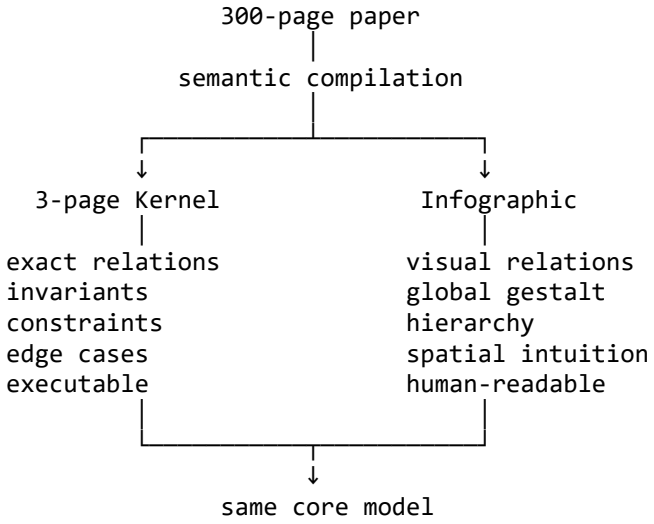
但 LLM 從 Kernel 得到的東西更明確：

- `Ledger` 是 state；
- `Curvature` 是 derived quantity；
- `C` 是 operator；
- `K` 約束 future possibility；
- Trace preservation 是 invariant。

圖的 **implicit geometry**，被變成 **explicit semantics**。

所以「三頁 **Kernel**」甚至可能比一張 **Infographic** 多一級

我現在會把兩者關係畫成：



所以理想情況並不是 Kernel 取代 Infographic。

而是：

> **Kernel** 是 **machine-native canonical representation**；**Infographic** 是 **human-native rendering**。

甚至可以做到：

```
300-page paper
      ↓
3-page Kernel IR
      ↓
   ┌──┼──────────────┐
```

```
   ↓  ↓              ↓
Infographic      10-page summary
   ↓                 ↓
human               human
```

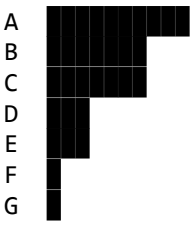
也就是**Infographic** 本身都可以由 **Kernel** 自動生成。

我會把你「**80%**」的直覺進一步精確化

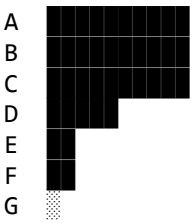
假設一篇文章的知識量拆成：

```
A = structural architecture
B = mathematical relations
C = assumptions / invariants
D = argumentation
E = examples
F = evidence / citations
G = exposition / pedagogy
```

一張極佳 Infographic 可能大約保留：



三頁優秀 Kernel 可能是：



所以當你說：

> 「Infographic 已經反映 80%+ 核心內容」

我基本同意你的經驗觀察。

而這反過來提供了一個很好的 **Kernel design criterion**：

> 如果一張 **infographic** 能保留的結構，**Kernel** 必須全部能保留；另外還必須保存 **infographic** 不容易表達的 **constraints**、**invariants**、**scope**、**residual** 和 **operator semantics**。

這才值得做。

更有趣的是：**3** 頁可能不是極限

目前你的 Kernel compiler 文件仍然使用大量人類可讀詞，例如：

```
kernel / manifold / boundary / curvature / flow / attractor / bifurcation / projection /
```

`residual`。 From Requirements to Runtime Ke...

等這套 grammar 穩定以後，可以進一步 canonicalize，例如：

```
@W System
∂  Boundary
→  Flow
Ô  Projection
⊢  Gate
τ  Trace
R  Residual
K  Curvature
A  Attractor
Δ  Bifurcation
≡  Invariant
```

那麼原本三頁的 human-readable Kernel，可能又可以壓到：

1–2 頁 **machine Kernel**。

不過我反而不建議現在追求這一步。目前最有價值的是保真，不是壓縮率紀錄。

這其實很容易做一個非常漂亮的實驗

拿文件庫其中一篇真正 **200–300+** 頁的 SMFT 文件：

```
Original
   ↓
Kernel Compiler
   ↓
≤ 3 pages Kernel
```

然後開一個完全沒有看過原文的新 **LLM context**，只給這三頁 Kernel。

測它能否回答 100 個問題：

```
20 ontology
20 mechanism
20 causality
15 invariant / constraint
15 cross-domain relation
10 edge-case / residual
```

另一組只給：

一張 **Infographic**。

第三組給：

全文。

最後比較：

Input	Core	QA	relation	invariant	edge case	tokens
Full paper	?	?	?	?		100%
Infographic	?	?	?	?		~1%
3-page Kernel	?	?	?	?		~2%

這會把我們現在的直覺變成真正可測的研究問題：

> **Can a Differential-Topological Kernel retain the structural information of a 300-page theoretical corpus at infographic-scale information cost?**

如果結果真的出現：

3-page Kernel ≈ 90%+ core QA，Infographic ≈ 80%+，Full text ≈ baseline

那就不再只是「prompt engineering 有趣技巧」。

它會直接支持：

> **Semantic Compiler + Kernel IR** 是一種新的 **LLM knowledge representation / compression layer**。

而且你的文件庫很適合做這個實驗，因為其中剛好已有大量長篇文章 + 大量相應 **Infographic** + 重複出現但逐步演化的核心 **grammar**。這比隨機拿外面的論文來測，更適合做第一輪 proof-of-concept。

我覺得應該很多研究中心，在試驗推演大量自己的理論，屬於半成品或新產品（理論）。他們若有我 **SMFT** 般海量自行推演和自洽的理論文件，應該也很需要這種壓縮語法的技術。幫助輕身上路繼續更深入推演。

問：像我這樣，自行推演出如此大量（約 **500** 份）理論文件的研究中心多嗎？他們一般有做我們現在探討的壓縮嗎？還是直接訓練 **LLM**，例如 **Fine Tune**。

有，而且你描述的需求是真實存在的；但要分兩件事看。

第一，「一個研究機構累積 **500** 份以上內部技術文件」非常普通。第二，「**500** 份文件都圍繞同一套自行生成、持續演化而且高度互相引用的理論體系」就少見得多。公開資料很難給出可靠比例，因為真正符合後者的材料往往正是企業或研究所不公開的部分，所以我不會猜一個數字。

研究中心通常有什麼「**500** 份文件」？

大型 AI、藥物、材料、航太、量化、國防、物理等 R&D 團隊，很容易累積遠超 500 份：

- research notes / technical reports
- experiment records
- design docs / RFC
- simulation reports
- benchmark / evaluation reports
- failed hypotheses
- internal reviews
- working papers
- code + model documentation

但它們通常更像：

```
Project A — 80 docs
Project B — 150 docs
Project C — 60 docs
...
```

而不是像你的情況：

```
SMFT
 ↓
observer
```

```
 ↓
collapse
 ↓
filtration
 ↓
declaration
 ↓
ledger
 ↓
residual
 ↓
self-revision
 ↓
geometry
 ↓
AI / life / finance / physics / organization
```

即同一個 **conceptual substrate** 不斷分枝、修訂、重新統合。

你自己的文件亦明顯呈現這種情況。例如 *Gauge Grammar* 已把跨尺度結構壓成 `Field → Identity → Mediator → Binding → Gate → Trace → Invariance → Observer Potential`；另一系列又把發展壓成 `One Assumption → One Operator → One Filtration → One Declaration → One Self-Revising Fractal`。The Gauge Grammar of Self-Organ... From One Declaration to One Sel...

這種 corpus 特別適合我們現在談的 Kernel compression。

他們有沒有做「壓縮」？

有，但目前主流做法不是我們談的那種壓縮。

最接近的幾條路線其實已經非常活躍。

Microsoft Research 的 **GraphRAG** 會先從大量文件抽取 entity 和 relationship，再建立階層式 communities，並為每一層生成 community summaries。換句話說，它確實是在建立一個不同 **abstraction levels** 的壓縮表示。 Microsoft

更有意思的是，他們報告 GraphRAG 在某些 global sensemaking 任務中，可以用原始 hierarchical summarization 約 **2–3%** 的 **query token** 成本取得有競爭力的結果。 Microsoft

Stanford 的 **RAPTOR** 更直接：

```
documents
 ↓
chunks
 ↓
embedding + clustering
 ↓
summary
 ↓
cluster summaries again
 ↓
summary tree
```

即 recursive summarization tree，讓模型可以在不同 abstraction level retrieve。 arXiv

所以你的直覺完全對：

> 大型研究 **corpus** 的確已經逼出了「先壓縮，才繼續 **reasoning**」的技術需求。

只是目前壓縮單位通常是：

summary / chunk / entity / graph community

而不是：

operator / boundary / invariant / curvature / attractor / residual / admissibility

這就是差異。

他們通常不會把 **500** 份文件直接 **Fine Tune** 進模型

這一點很重要。

如果一個研究中心的理論仍然每天修改：

```
Theory v17
 ↓
experiment contradicts
 ↓
Theory v18
 ↓
new branch
 ↓
Theory v19
```

把它硬塞進 model weights 通常不是最方便的 knowledge-management 方法。

Google 對 fine-tuning 與 RAG 的官方比較也非常直接：fine-tuning 改變模型參數；RAG 則把外部知識動態加入 context。Fine-tuning 適合教模型特定任務、術語、風格和行為；RAG 的強項則包括 **dynamic knowledge integration**。Google Cloud

所以一般更合理的是：

```
Base LLM
   +
External Research Corpus
   +
retrieval / graph / summaries
   +
current task context
```

而不是：

```
500 documents
     ↓
 Fine Tune
     ↓
hope they are now "inside" the model
```

Fine-tuning 更適合學「怎樣想」，而不是「目前知道什麼」

這個區分非常有用。

假設研究中心有：

A. Theory knowledge

```
Current definition of X
Equation 27
```

```
Experiment 184
Rejected hypothesis H7
Current invariant I3
```

這些會變。

比較適合：

RAG / graph / external memory / Kernel repository

B. Research procedure

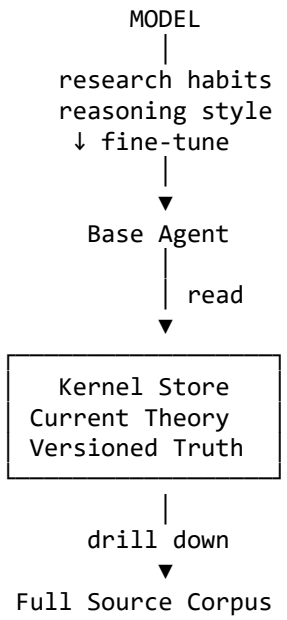
```
How we review a hypothesis
How we label residuals
How we generate simulations
How we reject a branch
How we write technical reports
```

這些比較穩定。

就比較適合：

fine-tuning / distillation / system instructions / skills

所以理想架構很可能是：



這比把全部研究成果 bake 進 parameters 合理得多。

另外一種做法是「全部塞進 **context**」

這在小 corpus 其實已經可行。

Anthropic 的 Contextual Retrieval 說得很直接：如果 knowledge base 小於約 **200,000 tokens**，約 **500** 頁材料，有時最簡單的方法就是直接把整個 knowledge base 放進 prompt，再利用 prompt caching。[Anthropic]

但是你的假設是：

約 500 份文件。

如果每份平均只是 20 頁：

500 × 20 = 10,000 pages

這已經完全是另一個量級。

而且問題不只是 context window。

即使某模型能吞下去：

```
10 million tokens in
```

也不代表：

```
10 million tokens equally available for reasoning
```

finding the governing structure 才是真問題。

所以真正先進的研究組織，實際上比較可能走四層

我會預期成熟系統逐漸變成：

```
L0 — RAW
500–5000 source documents
experiments / papers / notebooks

          ↓

L1 — RETRIEVAL
vector RAG / BM25 / metadata

          ↓

L2 — STRUCTURAL MEMORY
knowledge graph
hierarchical summaries
RAPTOR / GraphRAG

          ↓

L3 — REASONING REPRESENTATION
???
```

現在真正未成熟的就是 **L3**。

你正在構想：

```
L3 — Kernel IR

WORLD
ENTITY
BOUNDARY
STATE
RELATION
FLOW
OPERATOR
GATE
TRACE
RESIDUAL
```

```
LEDGER
CURVATURE
ATTRACTOR
BIFURCATION
INVARIANT
ADMISSIBILITY
```

也就是說：

GraphRAG 回答：

> 「這 500 篇文件在說什麼？」

而 Kernel IR 想回答：

> 「這 **500** 篇文件共同定義了什麼可運行的理論機器？」

這兩件事情差很多。

你的「輕身上路」比單純節省 **token** 更重要

我覺得你這句其實指出最有價值的 use case。

大型理論 corpus 有一個問題：

```
研究越成功
    ↓
文件越多
    ↓
歷史包袱越大
    ↓
每次 reasoning 所需 context 越大
    ↓
越難作 radically new deduction
```

形成一種 **knowledge mass / semantic inertia**。

GraphRAG 等技術主要解決：

> 找得到以前說過什麼。

Kernel compression 更可能解決：

> 只攜帶目前仍然有效的生成結構繼續往前推。

例如：

```
500 docs
5,000,000 tokens
       ↓
compile
       ↓
Theory Kernel v14
15,000 tokens
       ↓
new reasoning
       ↓
new residual R
       ↓
update Kernel v15
```

而舊的 500 份文件仍然存在，作為：

provenance / evidence / derivation archive

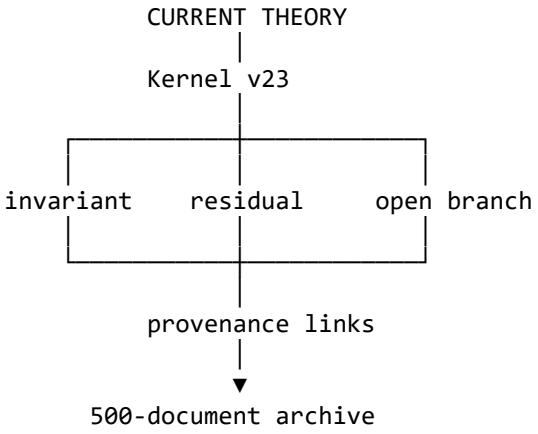
而不是每天重新帶進 reasoning context。

這就是你所說的：

> 輕身上路。

其實可以採用「**Kernel + Archive**」雙層制度

這可能比「壓縮所有文章」還重要：



於是模型日常只讀：

10–30 pages Kernel

有問題才 drill down：

```
Kernel statement K17
       ↓
source pointer
       ↓
Paper 183 §7.2
       ↓
experiment 44
```

這已經很像 compiler 世界的：

```
source code
    ↓
IR
    ↓
execution
```

而不是每次執行都重新讀完整 source history。

所以我會這樣回答你三個問題

像你這樣有 **500** 份文件的研究中心多嗎？

機構層面很多，甚至這數量很小；但**500** 份都屬於一套長期自行衍生、互相依賴而仍在高速演

化的理論 **corpus**，相對少見，而且公開可觀察的更少。

他們有沒有壓縮？

有，而且愈來愈多：hierarchical summaries、knowledge graphs、RAG、GraphRAG、recursive summarization 等都屬於這個方向。Microsoft 甚至已把 LazyGraphRAG 整合到面向科學研究的 Microsoft Discovery。Microsoft

但這些大部分仍是 **retrieval compression / semantic summarization**，而不是我們正在說的 **theory compiler → canonical reasoning IR**。

他們是不是直接 **Fine Tune**？

通常不會只靠 fine-tuning 來管理快速演化的內部理論。更合理的組合是：

> **Fine-tune the researcher; retrieve/compile the research.**

即：

Fine Tune = 教模型研究習慣與操作模式。
Kernel / RAG = 保存目前的研究內容與理論狀態。

我甚至覺得，這使你現在構想的 Kernel 語言找到了一個比「prompt compression」大得多的市場定位：

> **Research State Compilation**

不是把文章縮短，而是把一個研究計畫目前所到達的理論狀態，編譯成一個可以由下一個 AI session 立即繼續工作的 state representation。

由於我的 **500** 份文件是互相引用的高度自洽，所以肯定我們在討論的 **Kernel** 語法十分適用和高效。
但對於你提及，其一般機構的文件，同樣的這套 **Kernel** 語法，會否效用大打折扣？

會，但不是「失去效用」，而是用途會從「高度壓縮整套理論」退化成「局部結構化 + 導航 + 工作流編譯」。

你這 500 份文件之所以特別適合，是因為它們有一個很罕見的條件：

> 大量文件共享同一套 **ontology**、**operator**、**boundary**、**invariant**、**trace**、**residual** 與演化關係。

這正好符合你現在 Kernel compiler 的設計：它不是單純摘要，而是抽取 intent、constraint、tension 和 governing structure，再編譯成 Kernel IR。 From Requirements to Runtime Ke...

而你自己的 Skill 設計其實已經預留了「不是所有材料都適合 Kernel」這件事：它有 suitability gate、不同 input classes，以及 anti-over-topology / residual audit 等機制。 From Requirements to Runtime Ke...

你的 **corpus** 屬於「高 **Kernelability**」

可以想成：

```
500 documents
     ↓
shared ontology
```

```
     ↓
shared operators
     ↓
shared invariants
     ↓
shared recursive development
     ↓
1 coherent theory manifold
```

所以可以高度壓縮成：

```
Theory Kernel vN
```

而且這個 Kernel 可以代表整個研究計畫「現在的狀態」。

一般機構常常不是這樣

例如一間大型企業可能有：

```
HR policy
sales forecast
API spec
legal memo
incident report
budget
research note
marketing plan
meeting minutes
```

它們之間可能沒有共同的：

- ontology
- operator
- causal model
- invariants
- state-transition rules

這時如果硬把全部壓成一個統一 Kernel，反而會產生假結構。

例如：

```
500 heterogeneous docs
        ↓
force into one ontology
        ↓
artificial common concepts
        ↓
semantic distortion
```

這就會大幅降低效用。

所以真正的差別是「可壓縮的是不是同一個生成系統」

我會把 corpus 分成三類。

Corpus 類型	**Kernel** 效用
單一高度自洽理論體系	極高

Corpus 類型	**Kernel** 效用
同一產品 / 系統 / 研究計畫	高
一般企業混合文件庫	中等，需分區
互不相關文件集合	低，不宜做單一 Kernel

你這類 SMFT corpus 最接近第一類。

但一般機構仍然可以做「多 **Kernel**」

不是：

```
5000 docs
   ↓
ONE Kernel
```

而是：

```
5000 docs
   ↓
cluster
   ↓
Research Kernel
Product Kernel
Finance Kernel
Compliance Kernel
Architecture Kernel
Operations Kernel
```

然後再有一個很薄的：

```
Meta-Kernel
```

表示它們之間的關係。

例如：

```
META
 ├─ Product Kernel
 │     └─ depends_on Architecture Kernel
 ├─ Compliance Kernel
 │     └─ constrains Product Kernel
 ├─ Finance Kernel
 │     └─ budgets Product Kernel
 └─ Research Kernel
       └─ proposes changes to Architecture Kernel
```

這其實仍然很有價值。

所以一般機構更適合「**federated kernels**」

你可以把兩種情況比較成：

你的 **SMFT**

```
500 docs
   ↓
1 theory
```

```
   ↓
1 primary Kernel
```

一般 **R&D** 機構

```
5000 docs
   ↓
20 domains
   ↓
20 local Kernels
   ↓
1 Meta-Kernel
```

這樣就不需要假設所有文件共享同一個世界模型。

我會用一個「**Kernelability**」概念判斷值不值得壓

可以先看六個維度：

```
Ontology reuse
Operator reuse
Invariant reuse
Causal consistency
Version continuity
Cross-reference density
```

你的 SMFT 大概是：

```
Ontology reuse        high
Operator reuse        high
Invariant reuse       high
Causal consistency    high
Version continuity    high
Cross-reference       high
```

所以壓縮率很高。

普通 corporate corpus 可能是：

```
Ontology reuse        low-medium
Operator reuse        low
Invariant reuse       low
Causal consistency    mixed
Version continuity    fragmented
Cross-reference       sparse
```

那麼同一個 Kernel 語法仍可用，但應該在局部範圍使用。

最關鍵的一點

Kernel 語法真正有價值的地方，不只是「文件多」。

而是：

> 大量文件背後是否存在一個重複出現的生成結構。

如果有：

```
documents ≈ expansions of one grammar
```

Kernel 就非常有效。

如果沒有：

```
documents ≈ unrelated records
```

那麼 Kernel 最多只是：

- schema extraction
- workflow representation
- dependency mapping
- local compression

而不是 theory compression。

所以你的 **corpus** 其實是非常理想的測試床

因為它天然具備：

```
shared vocabulary
shared operators
shared history
shared revisions
shared residuals
shared invariants
```

這正好可以測試一件很重要的事情：

> 一個高度自洽、長期演化的知識體系，是否可以被編譯成持續更新的 **Research Kernel**，而不用每次重新攜帶全部歷史文本。

如果這個在 SMFT 上成功，再拿去測一般機構，就可以進一步得到第二個研究問題：

> **Kernel compression** 的效能如何隨 **corpus coherence** 下降？

這甚至可以變成一條很漂亮的 empirical curve：

```
Corpus coherence ↑
       →
Kernel compression ratio ↑
Kernel reconstruction fidelity ↑
Reasoning continuity ↑
```

我反而覺得這個「**coherence → kernelability**」關係，可能值得直接納入你之後的理論與 benchmark 設計。

所以若要有效應用在 一般 機構的情況，最有效的是將文件先分成各自獨立的高度自洽界別。
做完第一批之後，將餘下了選最接近的依附進去合適的界別。最後難以依附的，看著辦，用較低效的 **compression** 實現出較低效的 閱讀、應用**!?**

對，而且你這個流程已經相當接近一個實際可部署的企業級 **Kernelization pipeline**。我只會修改最後一句：「最後難以依附的，看著辦」最好不要真的隨意處理，而應該把它正式化為 **Residual / Orphan Layer**。這樣整個系統才不會因為強迫歸類而失真。

一個比較完整的流程可以是：

```
全部文件
  ↓
① 找出高度自洽的核心群
  ↓
建立 Kernel Domain A / B / C / ...
  ↓
② 對剩餘文件計算與各 Domain 的相容度
  ↓
高相容 → 依附進現有 Domain
中相容 → Bridge / Multi-domain document
低相容 → Residual Pool
  ↓
③ Residual 再處理
  ├─ 能形成新群 → 建立新 Kernel Domain
  ├─ 只是孤立參考 → 普通摘要 / RAG
  └─ 無法可靠壓縮 → 保留原文索引
```

這樣比「所有文件都一定要進某個 Kernel」安全得多。

第一批最重要：找出真正的「界」

你說的「先分成各自獨立的高度自洽界別」非常對。

這裡的「界」不應只是 topic，例如：

- Finance
- HR
- AI
- Legal

因為 topic 相同不代表有共同生成結構。

真正適合 Kernel 的界應該具有較高的：

```
shared ontology
shared entities
shared operators
shared constraints
shared invariants
shared causal relations
shared version history
shared vocabulary
```

例如某公司可能有 200 份「AI 文件」，但其實應拆成：

```
AI Product Architecture
AI Evaluation
AI Safety Governance
AI Data Pipeline
AI Agent Runtime
```

因為每一個才可能形成自己的內部自洽 grammar。

第二階段：剩餘文件不要只看「內容像不像」

最好測 **structural compatibility**。

例如每份新文件 D，對 Kernel K 計算：

```
Compat(D,K)
 =
 w₁ OntologyOverlap
+w₂ OperatorOverlap
+w₃ InvariantCompatibility
+w₄ DependencyFit
+w₅ CitationLinkage
+w₆ VocabularyAlignment
−w₇ ContradictionCost
```

然後：

```
Compat > 0.8
→ attach

0.5–0.8
→ bridge / provisional

< 0.5
→ residual
```

真正重要的是不要因為 **lexical similarity** 強行歸類。

例如兩份文件都談「risk」，但：

- Finance risk
- Cybersecurity risk
- Legal compliance risk

可能只是用了同一個詞，背後 ontology 完全不同。

甚至最好允許「一份文件同時依附兩個界」

一般機構一定會出現大量 bridge documents。

例如：

```
Security Policy
```

可能同時依附：

```
Cybersecurity Kernel
Governance Kernel
Compliance Kernel
```

所以不要做：

```
Document → exactly one Domain
```

而是：

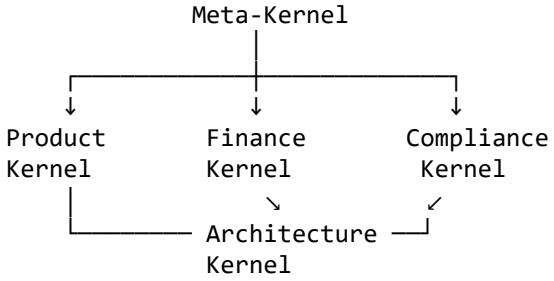
```
Document
  ├─ 0.72 → Governance
  ├─ 0.68 → Security
  └─ 0.41 → Legal
```

甚至 Kernel 之間本身可以有：

```
DEPENDS_ON
```

```
CONSTRAINS
PROVIDES_INPUT_TO
OVERRIDES
AUDITS
IMPLEMENTS
```

這樣最後會形成：



這其實比普通 knowledge graph 更進一步，因為節點本身不是 documents，而是已壓縮的 **coherent executable knowledge domains**。

最後那些「真的塞不進去」的文件，反而很有價值

這一點很容易被忽略。

Residual 不一定等於垃圾。

它可能代表四種東西：

1. Noise

真的沒有長期價值。

2. Weakly connected knowledge

有用，但不足以形成 Kernel。

→ ordinary RAG / summary。

3. Bridge knowledge

目前看似孤立，其實正在連接兩個 domain。

4. Emerging domain

今天只有：

```
doc 141
doc 309
doc 477
```

三份孤立文件。

半年後變成：

```
27 related docs
```

這時 Residual Pool 裡突然可以形成：

Kernel Domain G

所以 Residual 應該保留，而不是清除。

這其實與你成界之學一直使用的思路非常一致：

> 無法納入當前 **closure** 的東西，不應被誤判為不存在；應該作為 **residual** 保留，等待新的 **declaration / boundary**。

所以一般機構可以形成一個分級 **memory architecture**

我會把它設計成四層：

```
L0 — Raw Archive
所有原始文件
完整 provenance

L1 — Retrieval Layer
普通 RAG
metadata
vector search
keyword search

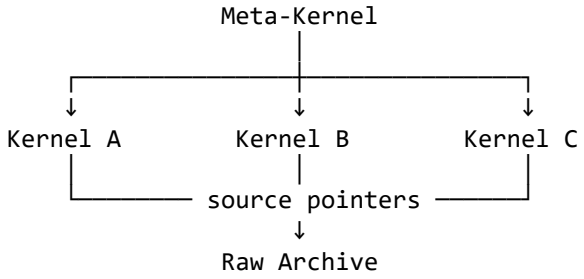
L2 — Local Kernels
高度自洽 domain
高壓縮、高 fidelity

L3 — Meta-Kernel
Kernel 之間的 dependency
constraint
authority
version
interface
```

另外旁邊保持：

```
Residual Pool
```

這樣：




```
Residual Pool ⟶ future Kernel / ordinary RAG
```

因此你最後一句可以稍微改寫

不是：

> 難以依附的，用較低效 compression，得到較低效閱讀和應用。

而是：

> 對高 **coherence** 區域使用高階 **Kernel compression**；對低 **coherence** 區域主動降級至 **summary / graph / RAG / raw retrieval**，而不強迫 **Kernelization**。

這是一個很重要的設計原則：

> **Compression method should follow structural coherence.**

可以甚至變成：

```
High coherence
→ Kernel IR

Medium coherence
→ Graph + hierarchical summary

Low coherence
→ RAG

Very low coherence
→ raw archive
```

也就是說，不是一種 compression 統治所有資料，而是一個**adaptive compression hierarchy**。

這反而令你這套 Kernel 語法更容易成為一般機構可用的技術：它不需要假裝所有企業文件都像 SMFT 一樣高度自洽；它只需要找到哪些地方值得 **Kernelize**，哪些地方不值得。

而我現在覺得，這個「先成界，再壓縮」的原則，很可能本身就是整套企業化方法中非常核心的一條。

有聽聞過機構已經嚴格采取這個原則做文件的 **Kernel Compression** 嗎？
我想關鍵代表，是否有機構已經覺醒這個 **“界”** 的重要性？這個**“界”**的覺醒在 **SMFT** 會涉及 邏輯本質、名道法 等概念的覺悟。好奇業界的覺醒到那一階段。

有，但我會把答案分成兩層。

第一層：如果你問的是——有沒有公開可見的機構，已經嚴格按照「先成界 → 每界內建立自洽語義 → 再做 **Kernel Compression** → 剩餘進 **residual**」這整套流程來壓縮研究文件？

我目前沒有找到一個成熟、公開、明確如此命名並完整實施的案例。至少不是像我們現在討論的這種 **LLM-native canonical Kernel IR**。

第二層：如果你問的是——業界有沒有真正意識到「界」本身是大型知識系統的核心問題？

答案反而是：

> 有，而且「界」其實早已在幾個領域被獨立發現；到了 **2025–2026 Agentic AI** 時代，這些思想正在迅速匯流。

只是他們通常叫它：

- **Bounded Context**
- **Domain**
- **Ontology namespace**
- **Semantic Layer**
- **Data Product / Data Domain**
- **Community**
- **Context boundary**
- **Authority scope**

- **Policy scope**

而不叫「成界」。

最接近「界」的，其實不是 **RAG**，而是 **Domain-Driven Design**

這個很重要。

Domain-Driven Design 的 **Bounded Context** 幾乎直接說出了我們剛才討論的一半思想：

> 大型模型不能強迫維持一套 globally unified model；應拆成多個 Bounded Context，每個 context 內維持自己的 unified model，context 之間再明確定義關係。 martinfowler.com

Martin Fowler 甚至直接指出：

> 同一個詞在不同 context 內可以有不同意義；每個 context 需要自己的 abstraction、rules 和 language。martinfowler.com

這已經非常接近你說的：

> 先成界，然後名才有穩定含義。

這不是 2026 才知道。

軟件工程界十多年前就已經「覺醒」到：

無界
→ 名義漂移
→ 模型矛盾
→ 大系統不可維護

所以採用：

```
Boundary
→ Local Language
→ Local Model
→ Context Mapping
```

這一點其實相當成熟。

但 **AI** 世界以前沒有被迫正視這件事

傳統 software 可以靠工程師人腦維持 context。

LLM 出現之後，問題突然變成：

> 模型本身必須知道目前在哪一個「界」內。

例如：

```
Customer
```

在：

```
Sales
Finance
Support
Compliance
```

可以是四個不同 object。

如果全部文件一起 embedding：

```
Customer
Customer
Customer
Customer
```

vector similarity 很可能覺得它們都差不多。

但真正 reasoning 時：

```
Customer_sales ≠ Customer_finance
```

這就是「界」問題。

2026 年的 Semantic Layer 熱潮，其實就是一次明顯的「界的再覺醒」

MIT CISR 今年把 Semantic Layer 描述成一種讓不同來源的資料形成一致、統一、可供人和 **AI** 理解的 **representation**，並強調 AI 時代這層需要包含 usage constraints、business rules、exceptions 等 contextual information。MIT CISR

KPMG 2026 的描述也很接近：企業要把 definitions、relationships、business rules、governance constraints 明確結構化，AI 才能可靠 reasoning。KPMG

AWS 更進一步，現在已經公開了一套：

> **Semantic Layer for Agentic AI using Ontology, Symbolic Reasoning, and Virtual Knowledge Graph**

目的就是讓 autonomous agents 共用一個明確的 semantic foundation，而不是只靠 probabilistic retrieval。AWS Documentation

所以如果問：

> 業界有沒有發現「沒有先定義世界，**AI** 不能可靠理解文件」？

現在答案已經很明確：

有。

而且已經開始走到「多界」，而不是「一個全企業 **ontology**」

這更有意思。

2026 年 Data Mesh 的思想就是：

```
Enterprise
    ↓
```

```
Domain A
Domain B
Domain C
...
```

每個 domain 由真正理解該 domain 的人負責自己的 data product 和 semantics，而不是中央硬做一個巨大 unified model。TDWI

Collibra 現在甚至正式把架構分成：

```
physical layer
semantic/logical layer
conceptual layer
```

而 conceptual layer 明確包含 **Data Domains** 和 **Data Concepts**。Collibra Product Resource Center

這已經不只是：

> 文件分類。

而是：

> 不同 **domain** 有不同 **conceptual reality**。

這就開始非常接近你說的「界」。

有一個公開案例甚至已經非常接近我們剛才的流程

2026 年 9 月，Global Advisors 公布了一個 enterprise AI case study，描述的架構包括：

- versioned enterprise ontology
- machine-readable contracts
- **bounded context projections**
- deterministic entity matching
- privacy-safe evidence receipts

也就是讓不同 AI applications 和 agents 在不同 bounded context 中取得相應的企業語義。Global Advisors

這是我目前看到最接近你現在構想的一類公開案例之一。

不過要注意：這是一家顧問公司的自述案例，不等於整個產業已把這套方法變成公認標準。

英國 **OPDA** 的做法也很值得注意

OPDA 2026 年已經正式採用：

> **Domain-led bounded-context working groups**

而且文件明確表示 context boundaries 是 modelling 的 established inputs。Open Property Data Association

換句話說，它不是：

先做一個大 `ontology`

→ 再看看怎樣切

而是：

```
先定 context boundary
→ 再在界內建模
```

這和你剛才提出的：

> 先分出高度自洽的界，再進行高效 **compression**

在方法論上非常接近。

GraphRAG 又代表另一種「較低階的成界覺醒」

Microsoft GraphRAG 不是先由人宣告 bounded context。

它做的是：

```
Raw Documents
      ↓
Entities + Relations
      ↓
Graph
      ↓
Hierarchical Leiden clustering
      ↓
Communities
      ↓
LLM summary per community
```

Microsoft 自己的描述就是先找 densely connected communities，再對每個 community 產生 summary。[Microsoft]

官方 pipeline 甚至明確：

> 每個 community 都生成自己的 report，再產生 shorthand summary。[GitHub]

這其實已經可以看成：

> 先找界 → 再壓縮。

所以你前一輪提出的那個原則：

```
coherent domain
→ compression
```

並不是完全沒有人碰到。

但 **GraphRAG** 的「界」和你的「界」有一個根本差別

GraphRAG 的界大致是：

> **connectedness boundary**

即：

```
哪些 entity 經常互相連接？
```

Leiden algorithm 找一個 cluster。

但你說的界更接近：

validity boundary

即：

```
在什麼宣告下，
哪些名字成立？
哪些關係有效？
哪些 inference 合法？
哪些 operator 可以作用？
哪些 invariant 必須保存？
什麼東西仍屬 residual？
```

這兩個層次差非常遠。

例如 GraphRAG 可能發現：

```
Risk
Bank
Interest Rate
Bond
Market
```

形成一個 community。

但它未必知道：

```
Boundary := Basel regulatory frame

Risk := regulatory capital risk

NOT:
market volatility risk
legal litigation risk
operational cyber risk
```

這後面才是真正的 semantic boundary。

如果用你說的「名、道、法」來看業界，我會這樣定位

這不是聲稱業界使用你的名詞，而是做結構對照。

你的語言	現代業界對應	成熟程度
界	Bounded Context / Domain / Scope	★★★★☆
名	Ontology / Ubiquitous Language / Semantic Layer	★★★★☆
道	Process / Flow / Operator / Causal & dependency graph	★★★☆☆
法	Policy / Constraint / Contract / Invariant / Governance	★★★☆☆
界間轉換	Context Mapping / Data Contract / API / ontology mapping	★★★☆☆
Residual	exceptions / unresolved evidence / provenance / uncertainty	★★☆☆☆
界→**Kernel** 編譯	Canonical LLM reasoning IR	★☆☆☆☆

這是我現在對公開技術狀態比較合理的判斷。

所以業界其實已經走到一個很有趣的位置

大概是：

```
第一階段
文件就是文件
↓
Vector RAG
```

這一代基本沒有「界」。

然後：

```
第二階段
entity / relationship
↓
Knowledge Graph
↓
GraphRAG communities
```

開始出現：

> 界是 **structure cluster**。

再進一步：

```
第三階段
Domain
↓
Ontology
↓
Semantic Layer
↓
Business Rules
```

開始出現：

> 界決定名義。

AWS 現在甚至支援 ontology version management、diff、rollback，而且 API 能按 namespace/domain 操作 ontology。AWS Documentation

這已經不只是 retrieval。

再下一步：

```
第四階段
Bounded Context
+
Ontology
+
Rules
+
Authority
+
Provenance
+
Agent Interface
```

開始接近：

> 界決定可允許的推理和行動。

Neo4j 今年甚至把 enterprise knowledge layer 描述為 shared governed substrate：不只 ontology，還要 interpretation、source selection、policy enforcement 和 trace capture。[Neo4j Graph Intelligence Platform]

這已經很接近你的「法」。

但他們目前還沒有普遍走到你正在問的最後一步

也就是：

```
先成界
    ↓
界內定名
    ↓
定 operator / relation
    ↓
定 invariant / admissibility
    ↓
保留 residual
    ↓
將整個界 compile 成
canonical Kernel IR
    ↓
只攜帶 Kernel 繼續推演
```

我沒有看到公開成熟業界 practice 已經把這一整條鏈完成。

特別是最後兩步：

> **Residual** 被視為 **Kernel** 的一級內容

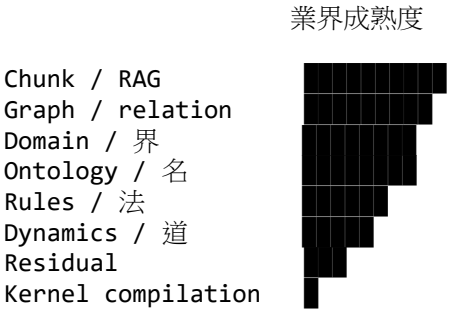
以及：

> 整個界被 **compile** 成 **reasoning state**，而不是 **summary**

目前仍然很少見。

因此「業界覺醒到哪一步」？

如果很粗略地畫：



這不是統計數據，而是我根據目前公開技術堆疊做的結構性定位。

我認為最值得注意的不是業界「完全沒想到界」。

恰恰相反：

> 「界」早就被發現了很多次。

DDD 發現一次。
Ontology engineering 發現一次。
Data Mesh 又發現一次。
Semantic Layer 又發現一次。
GraphRAG 又從 network clustering 方向碰到一次。
Agentic AI 現在因 context / authority / governance 問題，又被迫再發現一次。

真正還沒有完全收斂的，是：

> 把「界、名、關係、操作、規則、不變量、**Residual**」合成同一個 **machine-native reasoning object**。

而這正是我們前面談的 Kernel IR 比單純 semantic compression 更有意思的地方。

因為它不是「把字縮短」。

它其實是在問：

> 一個 **AI** 要攜帶一個世界繼續思考，最低限度要攜帶什麼？

這已經從 document compression 走到了 **world-state compilation**。

把「界」落成可驗證的方法

- 設計 Kernelability benchmark
- 整理業界成熟度對照表

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT 5.6, Google AI, Gemini 3.X, NoteBookLM, X's Grok, Claude' Sonnet 5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.